



[Security4] - Fundamental of Cryptography

Exercise 1

- (Cryptanalysis) Though encryption is primarily designed to preserve confidentiality and integrity of data, the mechanism itself is vulnerable to brute force (statistical analysis). In other words, the more we see the encrypted data, the easier we can hack it. In this exercise, you are asked to crack the following cipher text. Please provide the decrypted result and explain your strategy in decrypting this text.
- **Cipher text**
- PRCsoFQX FP QDR AFOPQ CZSPR LA JFPALoQSKR. QDFP FP ZK LIU BROJZK MOLTROE.
- a. Count the frequency of letters. List the top three most frequent characters.
 - **Answer:**
 - ('P', 7)
 - ('R', 6), ('O', 6), ('F', 6)
 - ('Q', 5)
- b. Knowing that this is English, what are commonly used three-letter words and two-letter words. Does the knowledge give you a hint on cracking the given text?
 - **Answer:**
 - คิดว่า QDR = the, FP = is
 - 'PRCSOFQX FP QDR AFOPQ CZSPR LA JFPALoQSKR. QDFP FP ZK LIU BROJZK MOLTROE.'
 - 'seCSOitX is the AiOst CZSsR LA JisALotSke. this is ZK LIU BeOJZK MOLTeOE.'
 - ZK = an
 - 'PRCSOFQX FP QDR AFOPQ CZSPR LA JFPALoQSKR. QDFP FP ZK LIUBROJZK MOLTROE.'
 - 'seCSOitX is the AiOst CaSsR LA JisALotSne. this is an LIUB eOJan MOLTeOE.'
 - seCSOitX = security
 - 'PRCSOFQX FP QDR AFOPQ CZSPR LA JFPALoQSKR. QDFP FP ZK LIUBROJZK MOLTROE.'
 - 'security is the Airst causR LA JisALrtSne. this is an LIU BerJan MrLTere.'
 - Airst = first, LA = of, R = e
 - 'PRCSOFQX FP QDR AFOPQ CZSPR LA JFPALoQSKR. QDFP FP ZK LIUBROJZK MOLTROE.'
 - 'security is the first cause of JisfortSne. this is an oIU BerJan MroTerE.'
 - เดราเอา
 - 'PRCSOFQX FP QDR AFOPQ CZSPR LA JFPALoQSKR. QDFP FP ZK LIUBROJZK MOLTROE.'
 - 'security is the first cause of misfortune. this is an old german proverb.'
- c. Cracking the given text. Measure the time that you have taken to crack this message.
 - **Answer:** 50 นาที
- d. Create a simple python program for cracking the Caesar cipher text using brute force attack. Explain the design and demonstrate your software.
(You may use an English dictionary for validating results.)

- **Answer:**

1. Pull file ที่เก็บ word ในภาษาอังกฤษไว้
2. ลองเลื่อนทีละ word ใน cypher text ไป i ลำดับตาม ASCII โดยถ้าเกิน Z ก็ทำการวนกลับมาที่ A (การ mode 26)
3. แบ่ง cypher text เป็นทีละ word (โดยการแบ่ง word ตาม ' ') → ทำการตรวจว่าคำที่ได้จาก split cypher text พบใน file dictionary ไหม

```
import datetime
ciphertext = 'PRCS0FQX FP QDR AFOPQ CZSPR LA JFPALOQSKR. QDFP FP ZK LIU BROJZK M
OLTROE.'
dictionary = []
answer = dict()
with open("Dictionary.txt", "r") as file:
    dictionary = file.read().split()

for i in range(0,26):
    temp_decrypted_sentence = ''
    for word in ciphertext:
        if word.isalpha():
            temp_decrypted_sentence += chr((ord(word) - ord('A') + i) % 26 + ord('A')).lower()
        else:
            temp_decrypted_sentence += word
    freq = 0
    for word in temp_decrypted_sentence.split():
        if word in dictionary:
            freq += 1

    answer[temp_decrypted_sentence] = freq

answer_sorted = sorted(answer.items(), key=lambda item: item[1], reverse=True)
for i in answer_sorted:
    print(i[0], " -> freq: " + str(i[1]))
```

```

PS D:\University_year3\Computer_Security\Security4_Cryptography> python .\main.py
yalbxozg oy zma joxyz libya uj soyjuxzbta. zmoz oy it urd kaxsit vxucaxn. -> freq: 4
sufvrta is tgu dirst fcvsu od misdortvnu. tgis is cn olx eurmcn prowurh. -> freq: 3
qsdtprgry gq res bgpqr datqs mb kgqbmprtls. regq gq al mjev cspkal nmpuspf. -> freq: 2
tvgsjyb jt uhv ejstu gdwv pe njtepsuov. uhjt jt do pmy fvsndo qspxvsi. -> freq: 2
prcsfoq fx qdr afopq czspr la jfpaloqskr. qdfr fp zk liu brojzk moltroe. -> freq: 1
wyjzvmxe mw xky hmvwx jgzwy sh qmwhsvxzry. xkmw mw gr spb iyvqgr tvsayvl. -> freq: 1
xzkawnyf nx ylz inwxy khaxz ti rnxitwasz. ylnx nx hs tqc jzwrhs uwtbzw. -> freq: 1
fhsievgv vf gth qvefg spifh bq zvfqbegiah. gtvf vf pa byk rhezpa cebjheu. -> freq: 1
hjukgxp xh ivj sxghi urkhj ds bxhsdgikcj. ivxh xh rc dam tjgbrc egdljgw. -> freq: 1
ikvlhyjq yi jwk tyhij vslik et cyitehjldk. jwyi yi sd ebn ukhcsd fhemkhx. -> freq: 1
npaqmdov dn obp ydmno axqnp jy hdnymqip. obdn dn xi jgs zpmhxi kmjrpmc. -> freq: 1
rteughsz hr sft chqrs eburt nc lhrcnqsumt. sfhr hr bm nkz dtqlbm oqnvtdg. -> freq: 0
umhxtkvc ku viw fktuv hexuw qf okufqtvxpw. viku ku ep qnz gwtoep rtqywtj. -> freq: 0
vxvylwd lv wjx gluvw ifyvx rg plvgruwq. wjlv lv fq roa hxupfq surzxuk. -> freq: 0
zbmcpah pz anb kpyza mjczb vk tpzkvyacub. anz pz ju vse lbytju wyvdybo. -> freq: 0
acndzqbi qa boc lqzab nkzdc wl uqalwzbdvc. boqa qa kv wtf mczukv xzweczp. -> freq: 0
bdoearcj rb cpd mrabc olebd xm vrbmxacewd. cprb rb lw xug ndavlv yaxfdaq. -> freq: 0
cepfbsdk sc dqe nsbcd pmfde yn wscnybdfxe. dqsc sc mx yvh oebwmz zbygebr. -> freq: 0
dfqgctel td erf otcde qngdf zo xtdozcegyf. ertd td ny zwi pfcxny aczhfcs. -> freq: 0
egrhdufm ue fsg pufed roheg ap yuepadfhzg. fsue ue oz axj qgdyoz bdaigdt. -> freq: 0
gitjfwho wg hui rfwgh tqjgi cr awgrcfhjbi. huwg wg qb czl sifaqb dfckifv. -> freq: 0
jlmwizkr zj kxl uzijk wtmjl fu dzjufikmel. kxzz zj te fco vldte gifnli. -> freq: 0
kmxngjals ak lym vajkl xunkm gv eakvgjlnfm. lyak ak uf gdp wnjuef hjgomjz. -> freq: 0
lmyokbmt bl mzn wbkml yvoln hw fblwhkmogn. mzbk bl vg heq xnkfvq ikhpnka. -> freq: 0
mozplcnu cm nao xclmn zwpmo ix gcmxilmpho. nacd cm wh ifr yolgwh jliqolb. -> freq: 0
oqbrnepw eo pcq zenop byroq kz ieozknprjq. pceo eo yj kht aqniy lnsqnd. -> freq: 0
PS D:\University_year3\Computer_Security\Security4_Cryptography>

```

Exercise 2

- (Cryptanalysis on Symmetric Encryption) Vigenère is a complex version of the Caesar cipher. It is a polyalphabetic substitution.
- a. Please review Kasiski examination Explain how it can be used to attack Vigenère.
 - **Answer: Kasiski examination:** แนวทางการแก้การencryptแบบ Vigenère cipher → คำซ้ำกัน & ห่างเป็นจำนวนเท่า → คำนี้ถูกเลื่อนด้วยตัวอักษรของกุญแจชุดเดียวกันเสมอ → Solve กลุ่มอักษรแบบ ciphertext
 - **Analysis Mechanism**
 - **ค้นหากลุ่มคำซ้ำ:** สแกนหาลำดับตัวอักษรใน "ciphertext" ที่ปรากฏซ้ำตั้งแต่ 3 ตัวอักษรขึ้นไป (เช่น Trigrams) เพื่อคัดกรองการซ้ำกันโดยบังเอิญออกไป
 - **วัดระยะห่าง (Interval):** บันทึกตำแหน่งดัชนีเริ่มต้นของกลุ่มคำซ้ำแต่ละคู่ แล้วคำนวณผลต่างระยะห่างระหว่างจุดที่พบ
 - **หาตัวประกอบร่วม (Common Factors):** นำระยะห่างทั้งหมดมาแยกตัวประกอบและคำนวณหาค่า "greatest common divisor" ตัวหารร่วมที่พบซ้ำบ่อยที่สุดมักจะตรงกับความยาวของคีย์ลับ m
 - **ลดรูปเพื่อถอดรหัส:** เมื่อได้ความยาวกุญแจ m แล้ว ข้อความจะถูกแบ่งออกเป็น m ชุดย่อย โดยแต่ละชุดย่อยจะมีสถานะลดรูปลงมาเป็น "Caesar cipher" แบบตัวอักษรเดี่ยว ซึ่งสามารถใช้ "frequency analysis" ภูคินกุญแจแต่ละตำแหน่งได้ทันที
 - **Step-by-Step Example:** สมมติมีระบบเข้ารหัสที่ใช้ข้อความและกุญแจลับดังต่อไปนี้:
 1. **ข้อมูลนำเข้าและการเข้ารหัส**
 - **Key:** กำหนดให้กุญแจลับคือคำว่า CAT (ความยาวกุญแจ $m = 3$)
 - **Plaintext:** สมมติว่ามีคำว่า THE ปรากฏ 2 ครั้งในข้อความ
 - ครั้งแรก ปรากฏที่ตำแหน่งตัวอักษรที่ 0
 - ครั้งที่สอง ปรากฏที่ตำแหน่งตัวอักษรที่ 18

```

Index:      0  1  2  ... 18 19 20 ...
Plaintext:  T  H  E  ... T  H  E  ...
Keystream:  C  A  T  ... C  A  T  ...
Ciphertext: V  H  X  ... V  H  X  ...

```

- ที่ตำแหน่ง 0: อักษร **T, H, E** ถูกจับคู่กับกุญแจ **C, A, T** ได้ผลลัพธ์เป็น **V, H, X**
- ที่ตำแหน่ง 18: เนื่องจาก 18 หารด้วย 3 ลงตัวพอดี ตัวอักษร **T, H, E** จึงวนรอบมาจับคู่กับกุญแจ **C, A, T** อีกครั้ง ทำให้ได้ผลลัพธ์เป็น **V, H, X** เช่นเดียวกัน

2. **การค้นหาและวัดระยะห่างบน Ciphertext:** ในมุมมองของผู้โจมตี จะไม่ทราบทั้ง **"plaintext"** และ **"key"** แต่จะตรวจพบกลุ่มตัวอักษรซ้ำ **VHX** สองจุดบน **"ciphertext"**:

- พบ **VHX** จุดแรกที่ตำแหน่งดัชนี = 0
- พบ **VHX** จุดที่สองที่ตำแหน่งดัชนี = 18
- คำนวณระยะห่างระหว่างตำแหน่ง: $18 - 0 = 18$

สมมติว่าผู้โจมตีสแกนพบคำซ้ำอีกกลุ่มหนึ่ง เช่น **KMB** ที่ปรากฏห่างกันเป็นระยะ **30** ตัวอักษร

3. **การหาตัวประกอบเพื่อสรุปความยาวกุญแจ (Key Length)**

นำระยะห่างที่วัดได้ทั้งหมดมาวิเคราะห์หาตัวประกอบ:

- ตัวประกอบของ 18 ได้แก่: 1, 2, **3**, 6, 9, 18
- ตัวประกอบของ 30 ได้แก่: 1, 2, **3**, 5, 6, 10, 15, 30

ตัวประกอบร่วมที่มีนัยสำคัญคือ **3** และ **6** โดยตัวเลข **3** เป็นตัวหารร่วมที่มีความเป็นไปได้สูงที่สุด ผู้โจมตีจึงตั้งสมมติฐานว่ากุญแจมีความยาวเท่ากับ 3 ตัวอักษร

4. **การแยกกลุ่มเพื่อถอดรหัสด้วย Frequency Analysis**

เมื่อทราบว่ากุญแจยาว 3 ตัวอักษร จะแบ่งตัวอักษรใน **"ciphertext"** ออกเป็น 3 กลุ่มอิสระ:

- **ชุดที่ 1:** นำตัวอักษรที่ตำแหน่ง 0, 3, 6, 9, 12, ... มารวมกัน (ทั้งหมดถูกเลื่อนด้วยอักษรตัวที่ 1 ของคีย์)
- **ชุดที่ 2:** นำตัวอักษรที่ตำแหน่ง 1, 4, 7, 10, 13, ... มารวมกัน (ทั้งหมดถูกเลื่อนด้วยอักษรตัวที่ 2 ของคีย์)
- **ชุดที่ 3:** นำตัวอักษรที่ตำแหน่ง 2, 5, 8, 11, 14, ... มารวมกัน (ทั้งหมดถูกเลื่อนด้วยอักษรตัวที่ 3 ของคีย์)

แต่ละชุดย่อยจะถูกนำไปนับสถิติความถี่ตัวอักษรเทียบกับภาษาอังกฤษปกติ ตัวอักษรที่มีความถี่สูงสุดในแต่ละกลุ่มจะตรงกับตัวอักษร E ทำให้คำนวณย้อนกลับได้ทันทีว่าอักษรของกุญแจแต่ละตำแหน่งคือ **C, A,** และ **T** ตามลำดับ

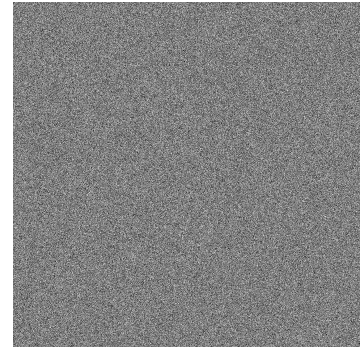
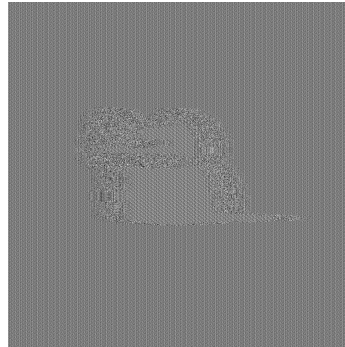
Exercise 3

- (Mode in Block Cipher) Block Cipher is designed to have more randomness in a block. However, an individual block still utilizes the same key. Thus, it is recommended to use a cipher mode with an initial vector, chaining or feedback between blocks. This exercise will show you the weakness of Electronic Code Book mode which does not include any initial vector, chaining or feedback.
 - f. What does the result suggest about the mode of operation in block cipher? Please provide your analysis
- **Answer:**
 - **ECB (Electronic Codebook):** เป็นการเข้ารหัสทีละบล็อกและแบ่งเป็นBlockเท่ากัน จึงทำให้Blockที่เหมือนกันจะให้ผลเหมือนกัน จึงจะได้ตรงว่า
 - พื้นหลังจะเหมือนกันหมด
 - แต่พอมา Encryptตรงขอบของตัวแมว จะให้textureของ block ที่แตกต่างกันไปทำให้เราเห็นขอบของแมว
 - ส่วนลำตัวตรงกลางที่มีสีเหมือนกัน ทำให้มีtextureเหมือนกัน
 - **CBC (Cipher Block Chaining):** การจุดเด่นที่มีการ Chaining encryption คือการที่Blockปัจจุบันเป็นผลจาก blockก่อนหน้า จึงจะเห็นได้ว่าแทบทุกส่วนของรูปภาพไม่มี pattern และดูคล้ายๆกัน เพราะมี noise จากblockหลังมาช่วย

Original

AES-256-ECB

AES-256-CBC



Exercise 3

- (Encryption Protocol - Digital Signature)
 - a. [Measure the performance of a hash function \(sha1\), RC4, Blowfish and DSA. Outline your experimental design.](#)
(Please use OpenSSL for your measurement)
 - **Answer:**
 - **Data Collection & Analysis:** Run 5 รอบ → หา Mean
 - **Independent Variables:** ประเภทของอัลกอริทึม (SHA-1, RC4, Blowfish, DSA), ขนาดของ "buffer size" (16B ถึง 8KB) และขนาดของ "key size" (เช่น 1024-bit, 2048-bit)
 - **Dependent Variables:** Symmetric → throughput (MB/s) & Asymmetric → operations per second
 - **Controlled Variables:** Hardware, OpenSSL version
-
- **RC4 และ SHA-1:** throughput ค่อนข้างสูงมาก
 - **Blowfish:** throughput กลางๆ เนื่องจากการประมวลผล block และการทำ chaining
 - **DSA:** throughput ค่อนข้างสูง เพราะ เป็นแบบ Asymmetric

```
PS D:\University_year3\Computer_Security\Security4_Cryptography> python .\ex4.py
Creating 10 MB dummy file...
Measuring SHA-1...
Measuring RC4...
Measuring Blowfish (CBC)...
Preparing DSA keys and measuring Sign/Verify...

=====
Algorithm      | Measured Performance
=====
SHA-1          | 364.38 MB/s (0.0274 s)
RC4            | 249.43 MB/s (0.0401 s)
Blowfish       | 106.91 MB/s (0.0935 s)
DSA Sign       | 0.0276 s (36.25 ops/s)
DSA Verify     | 0.0253 s (39.47 ops/s)
=====
PS D:\University_year3\Computer_Security\Security4_Cryptography> |
```

- b. [Comparing performance and security provided by each method.](#)

- **Answer:**
 - **Performance:** SHA-1 > RC4 > Blowfish > DSA
-
- **Security Status:**
 - **SHA-1 ("hash function"):** Insecure เนื่องจาก collision ของoutput เมื่อinput ต่างกันได้ง่าย
 - **RC4 ("stream cipher"):** Insecure การสุ่ม keystream(Biases)
 - **Blowfish ("block cipher"):** Legacy / Weak มีขนาดblock ข้อมูลเพียง 64 bits → เล็กเกินไป

- **DSA ("asymmetric digital signature")**: Conditionally Secure ปลอดภัยเมื่อใช้keyขนาด 2048 bits ขึ้นไป
- c. Explain the mechanism underlying Digital Signature. How does it combine the strength and weakness of each encryption scheme?

- **Answer:**

- 1. **Mechanism:**

- Sender: Message ผ่าน hash function เพื่อให้ได้ message digest ที่ยาวคงที่ → นำ message digest มา hash ด้วย Private key ได้ Digital signature
 - receiver:
 - เมื่อได้รับเอกสารและลายเซ็นดิจิทัล จะดำเนินการ 2 ส่วนขนานกัน:
 1. นำเอกสารต้นฉบับมาคำนวณหา "digest" ใหม่ด้วยตัวเอง
 2. นำลายเซ็นดิจิทัลมาถอดรหัสด้วย **Public Key** ของผู้ส่ง เพื่อกู้คืน "digest" ดั้งเดิม
 - เปรียบ Digest ทั้ง 2 อันที่ได้ตรงกันไหม

- 2. **Strengths**: Asymmetric มีแนวคิด Public key → key distribution ได้อย่างปลอดภัย, hash มีความเร็วสูงสำหรับข้อมูลจำนวนมาก

- 3. **Weaknesses**: Asymmetric ช้าเมื่อมีการเข้ารหัสที่ขนาดใหญ่, hash ไม่มีแนวคิดแบบ key → Identify ไม่ได้

- 4. **Combine the strength and weakness:**

- ใช้ hash function บีบอัดข้อมูลขนาดใหญ่ให้กลายเป็น message digest ที่มีขนาดคงที่และมีขนาดเล็ก เพื่อตัดปัญหาความล่าช้าในการคำนวณของ asymmetric encryption
 - ใช้ private key เพื่อสร้างเป็น digital signature